

고급머신러닝 보고서

활성화함수

Activation Function

전자공학부 전자공학과
2020144005 김윤성
2024. 10. 01 제출

목차 INDEX

01 활성화함수란

- 신경망
- 인공신경망
- Sigmoid
- ReLU
- Leaky ReLU
- Softmax

02 실습 #1)

- ReLU 구현
- 코드 분석
- 실행 결과
- 분석 내용

03 실습 #2)

- Softmax & CEE
- 코드 분석
- 실행 결과
- 분석 내용

04 실습 #3)

- DNN by keras
- 코드 분석
- 실행 결과
- 분석 내용

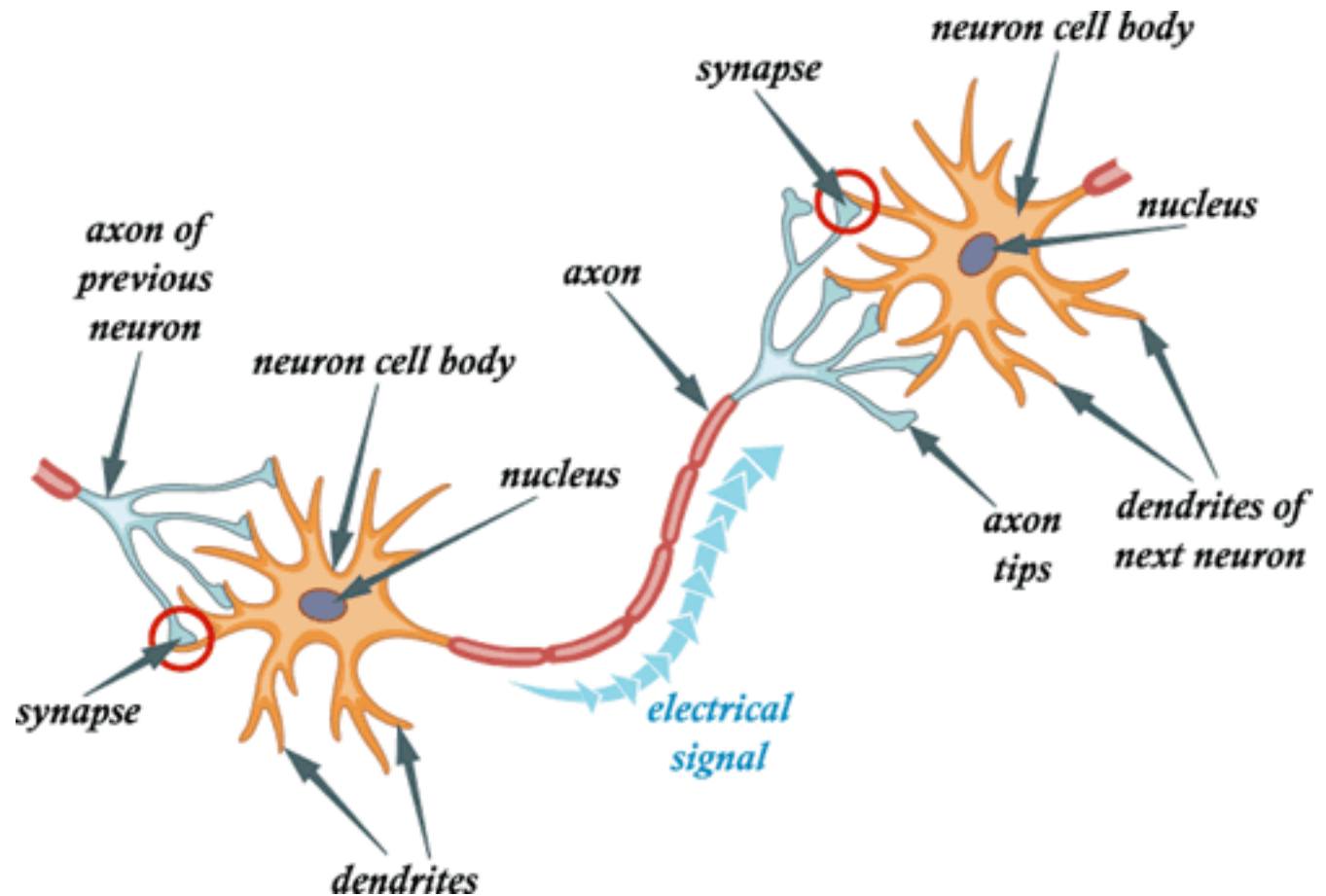
01

활성화함수란

Activation Function

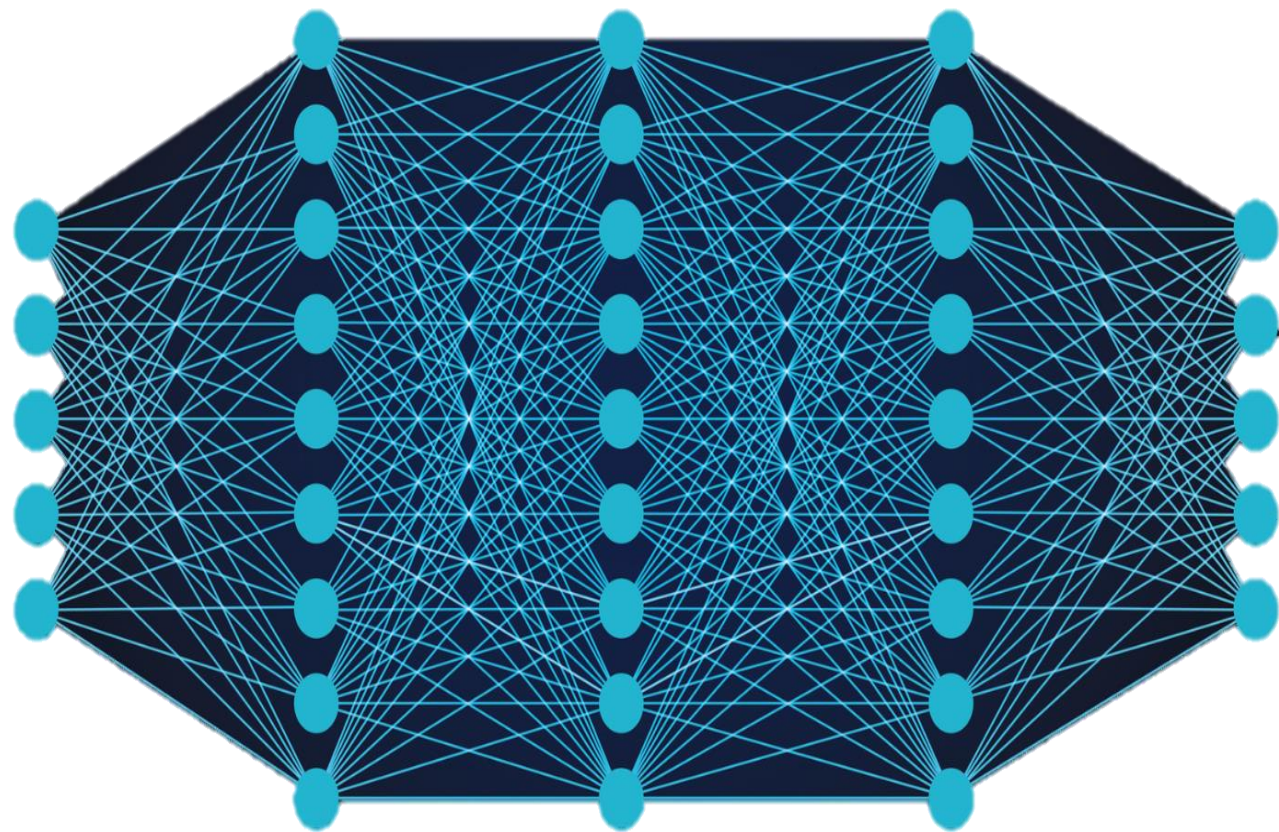
신경망 Neural Network

- 신경 세포(뉴런)의 집합체
- 전기적 신호로 다른 뉴런과 소통
- 시냅스에서 화학적인 신호로 변화
- 자극에 반응
- 학습, 기억 형성에 중요한 역할
- 신호의 세기에 따라 활성화/비활성
- 활성화함수로 뉴런의 출력 조절 가능



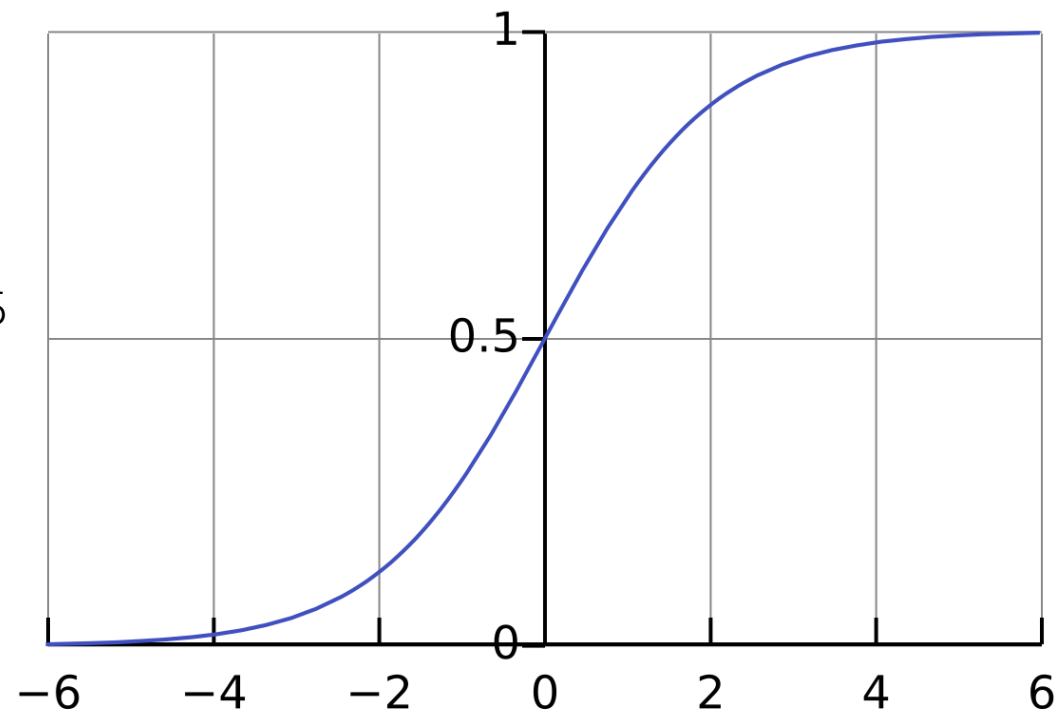
인공신경망 Artificial Neural Network

- 퍼셉트론이 모여 만들어진 신경망
- 순전파와 역전파를 통해 스스로 오차 조정, 최적화
- 비선형 관계 모델링 가능
- 데이터를 기반으로 한 학습 능력
- 새로운 데이터에 대한 예측이 가능
- NLP, 이미지 인식 등 다양한 분야에서 활용 가능
- 활성화함수로 뉴런의 출력 조절 가능



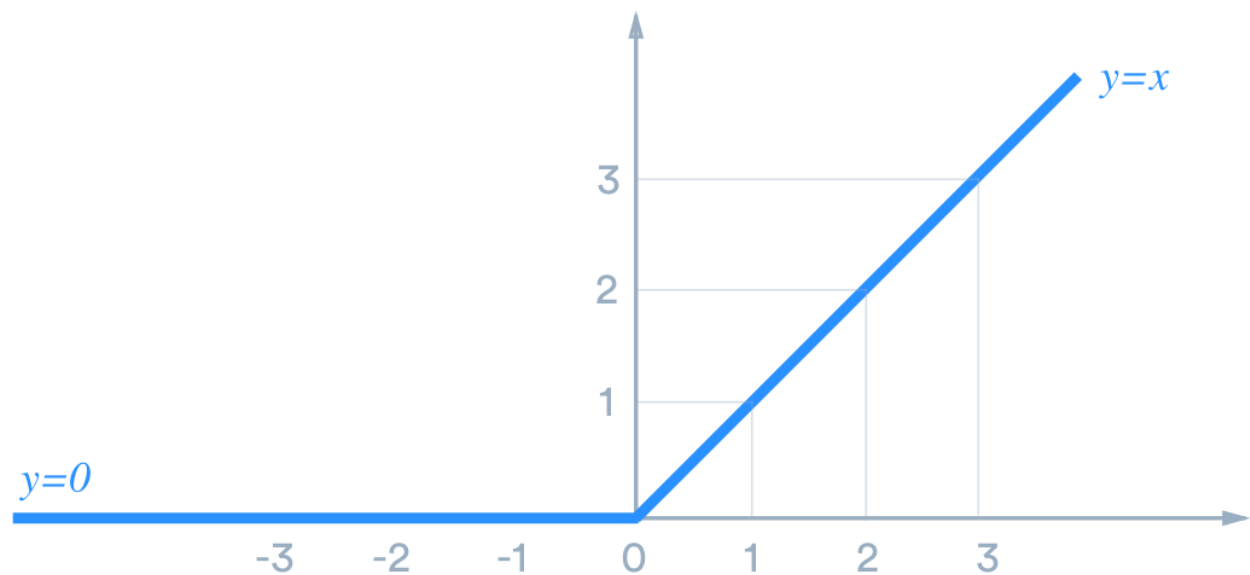
Sigmoid

- 출력: 0~1
- 이진 분류 (Binary Classification)에 주로 이용됨
- $Sigmoid(x) = y, \frac{d Sigmoid}{d x} = y(1 - y)$, 즉 미분값의 최대값은 0.25
- 1보다 작은 값이 계속 곱해짐 -> 0에 수렴
- 기울기 소실 (Gradient Vanishing) 발생 가능



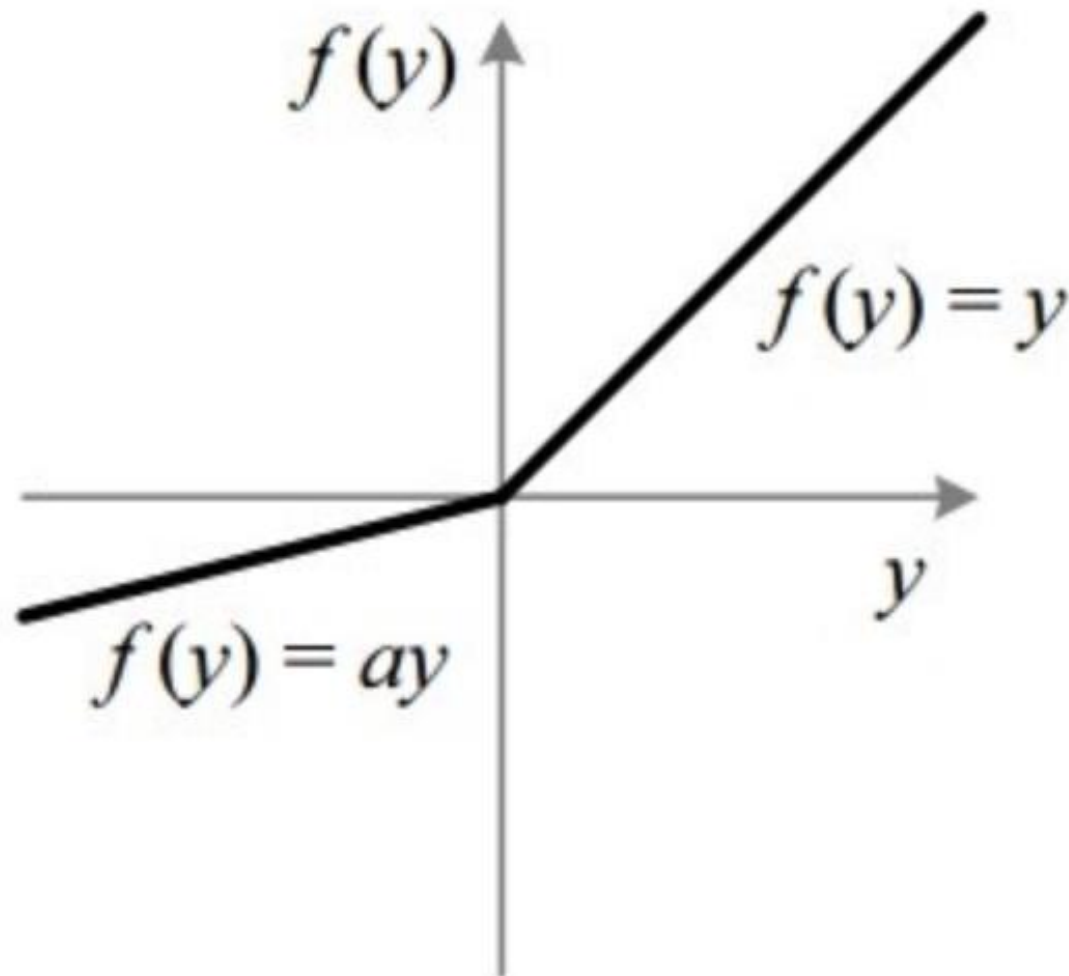
ReLU

- 출력: 0보다 크면 그대로, 0보다 작으면 0으로 출력
- 비선형성 우수, 계산 간편
- 0일때 미분이 되지 않으나 코드화가 쉬워 널리 사용됨
- 일부 뉴런이 비활성화되는 Dying ReLU 문제 발생 가능



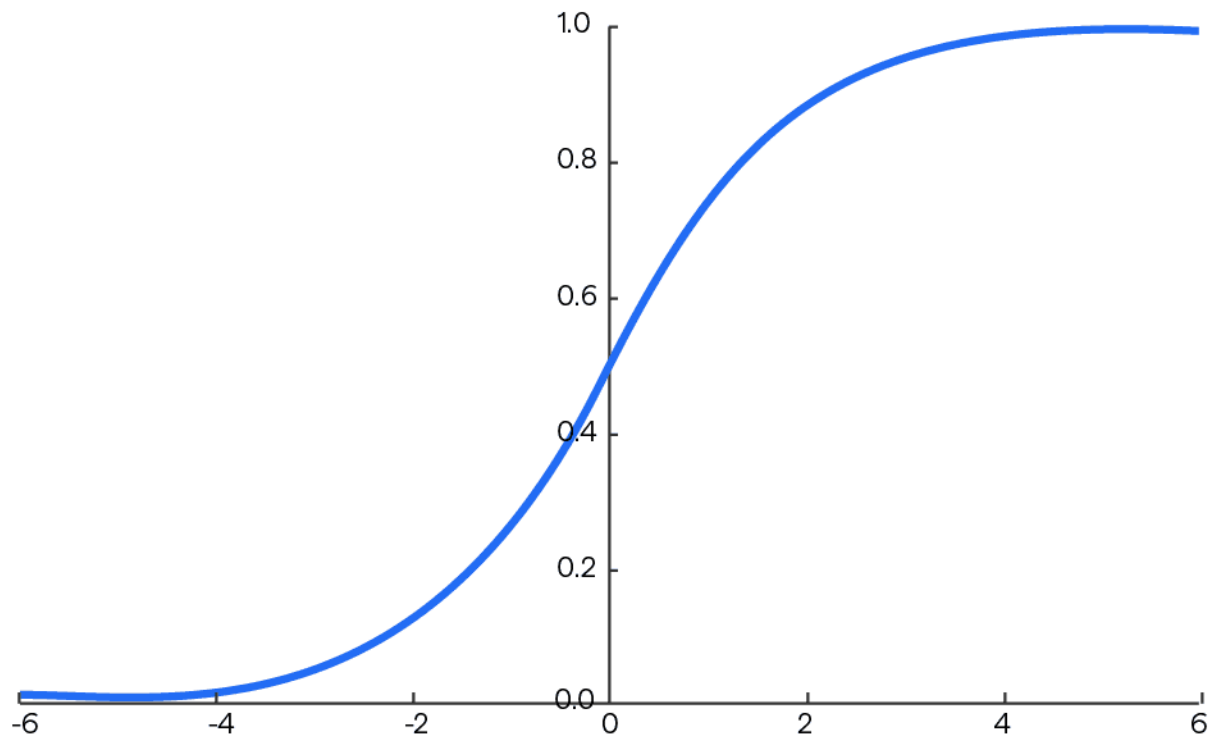
Leaky ReLU

- 출력: 0보다 크면 그대로, 0보다 작으면 0으로 출력
- 비선형성 우수, 계산 간편
- 0일때 미분이 되지 않으나 코드화가 쉬워 널리 사용됨
- Dying ReLU 문제 어느정도 해결 가능
- a값은 조절 가능



Softmax

- 각 클래스에 대한 출력값의 총 합이 1
- 각 클래스에 대한 확률값 계산 가능
- 주로 분류 문제의 출력층에서 사용



02

실습 #1

ReLU 구현

실습 #1) ReLU 구현

- Chap.1 실습 #2)에서 출력층을 제외한 모든 활성화 함수를 ReLU로 변경
- Chap.1 실습 #2)의 결과와 Chap.2 실습 #1)의 결과를 비교 분석
- 정확도 비교 이외에도 여러 분석 내용 기재

02 실습 #1 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

필요한 모듈 임포트

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_d(x):
    return x * (1 - x)

def One_Hot_Encoding(y, number_of_class):
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot

def ReLU(arr):
    return np.where(arr > 0, arr, 0)

def ReLU_d(arr):
    return np.where(arr > 0, 1, 0)
```

필요한 함수들을 정의한다.

ReLU(): 0보다 작으면 0, 0보다 크면 입력값을 그대로 출력

ReLU_d(): 0보다 작으면 0, 0보다 크면 1로 출력, 0이면 0으로 출력되게 함

02 실습 #1 코드 분석

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
Loss_history_relu = []
Accuracy_history_relu = []
Loss_history_sigmoid = []
Accuracy_history_sigmoid = []
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

```
input_size = X.shape[1]
hidden_size = 5 # 히든 레이어의 노드 수
output_size = len(np.unique(y)) # 출력 클래스 수
learning_rate = 0.1
epochs = 10000
graph_scale = 10 # 그래프의 점을 얼마나 촘촘하게 찍을것인지, int (숫자가 작아질수록 촘촘함)
```

Hyper Parameter들을 정의

02 실습 #1 코드 분석

```
weights_sigmoid = []
np.random.seed(42) # 항상 똑같은 결과를 얻기 위해 시드 고정
weights_sigmoid.append(np.random.randn(input_size, hidden_layer_sizes[0])) # 입력 -> 첫 번째 히든레이어의 가중치
for i in range(1, num_hidden_layers):
    weights_sigmoid.append(np.random.randn(hidden_layer_sizes[i - 1], hidden_layer_sizes[i])) # 각 히든레이어 사이의 가중치
weights_sigmoid.append(np.random.randn(hidden_layer_sizes[-1], output_size)) # 히든레이어 -> 출력 사이의 가중치

for epoch in range(epochs):
    # Forward Propagation
    layer_inputs = [X]
    layer_outputs = []
    for i in range(num_hidden_layers):
        input_to_layer = np.dot(layer_inputs[-1], weights_sigmoid[i])
        output_from_layer = sigmoid(input_to_layer)
        layer_inputs.append(input_to_layer)
        layer_outputs.append(output_from_layer)
    final_input = np.dot(layer_outputs[-1], weights_sigmoid[-1])
    final_output = sigmoid(final_input)
    # 오차 구하기 & 그래프를 그리기 위한 밀작업
    error = y_one_hot - final_output
    if epoch % 1000 == 0:
        print(f'Sigmoid Epoch {epoch}, Loss: {np.mean(np.abs(error))}')
    if epoch % graph_scale == 0:
        Loss_history_sigmoid.append(np.mean(np.abs(error)))
        y_pred = np.zeros_like(final_output)
        y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
        accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
        Accuracy_history_sigmoid.append(accuracy)
    # Backward Propagation
    d_output = error
    d_hidden = [d_output]
    for i in range(num_hidden_layers - 1, -1, -1):
        d_hidden_layer = np.dot(d_hidden[0], weights_sigmoid[i + 1].T) * sigmoid_d(layer_outputs[i])
        d_hidden.insert(0, d_hidden_layer)
    weights_sigmoid[-1] += np.dot(layer_outputs[-1].T, d_output) * learning_rate_sigmoid
    for i in range(num_hidden_layers - 1, 0, -1):
        weights_sigmoid[i] += np.dot(layer_outputs[i].T, d_hidden[i]) * learning_rate_sigmoid
    weights_sigmoid[0] += np.dot(X.T, d_hidden[0]) * learning_rate_sigmoid
```

02 실습 #1 코드 분석

```
weights_relu = []
np.random.seed(42) # 항상 똑같은 결과를 얻기 위해 시드 고정
weights_relu.append(np.random.randn(input_size, hidden_layer_sizes[0])) # 입력 -> 첫 번째 히든레이어의 가중치
for i in range(1, num_hidden_layers):
    weights_relu.append(np.random.randn(hidden_layer_sizes[i - 1], hidden_layer_sizes[i])) # 각 히든레이어 사이의 가중치
weights_relu.append(np.random.randn(hidden_layer_sizes[-1], output_size)) # 히든레이어 -> 출력 사이의 가중치

for epoch in range(epochs):
    # Forward Propagation
    layer_inputs = [X]
    layer_outputs = []
    for i in range(num_hidden_layers):
        input_to_layer = np.dot(layer_inputs[-1], weights_relu[i])
        output_from_layer = ReLU(input_to_layer)
        layer_inputs.append(input_to_layer)
        layer_outputs.append(output_from_layer)
    final_input = np.dot(layer_outputs[-1], weights_relu[-1])
    final_output = sigmoid(final_input)
    # 오차 구하기 & 그래프를 그리기 위한 밀작업
    error = y_one_hot - final_output
    if epoch % 1000 == 0:
        print(f'ReLU Epoch {epoch}, Loss: {np.mean(np.abs(error))}')
    if epoch % graph_scale == 0:
        Loss_history_relu.append(np.mean(np.abs(error)))
        y_pred = np.zeros_like(final_output)
        y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
        accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
        Accuracy_history_relu.append(accuracy)
    # Backward Propagation
    d_output = error
    d_hidden = [d_output]
    for i in range(num_hidden_layers - 1, -1, -1):
        d_hidden_layer = np.dot(d_hidden[0], weights_relu[i + 1].T) * ReLU_d(layer_outputs[i])
        d_hidden.insert(0, d_hidden_layer)
    weights_relu[-1] += np.dot(layer_outputs[-1].T, d_output) * learning_rate_relu
    for i in range(num_hidden_layers - 1, 0, -1):
        weights_relu[i] += np.dot(layer_outputs[i - 1].T, d_hidden[i]) * learning_rate_relu
    weights_relu[0] += np.dot(X.T, d_hidden[0]) * learning_rate_relu
```

Chap.1의 실습 코드에서 Sigmoid 함수를 ReLU로 교체

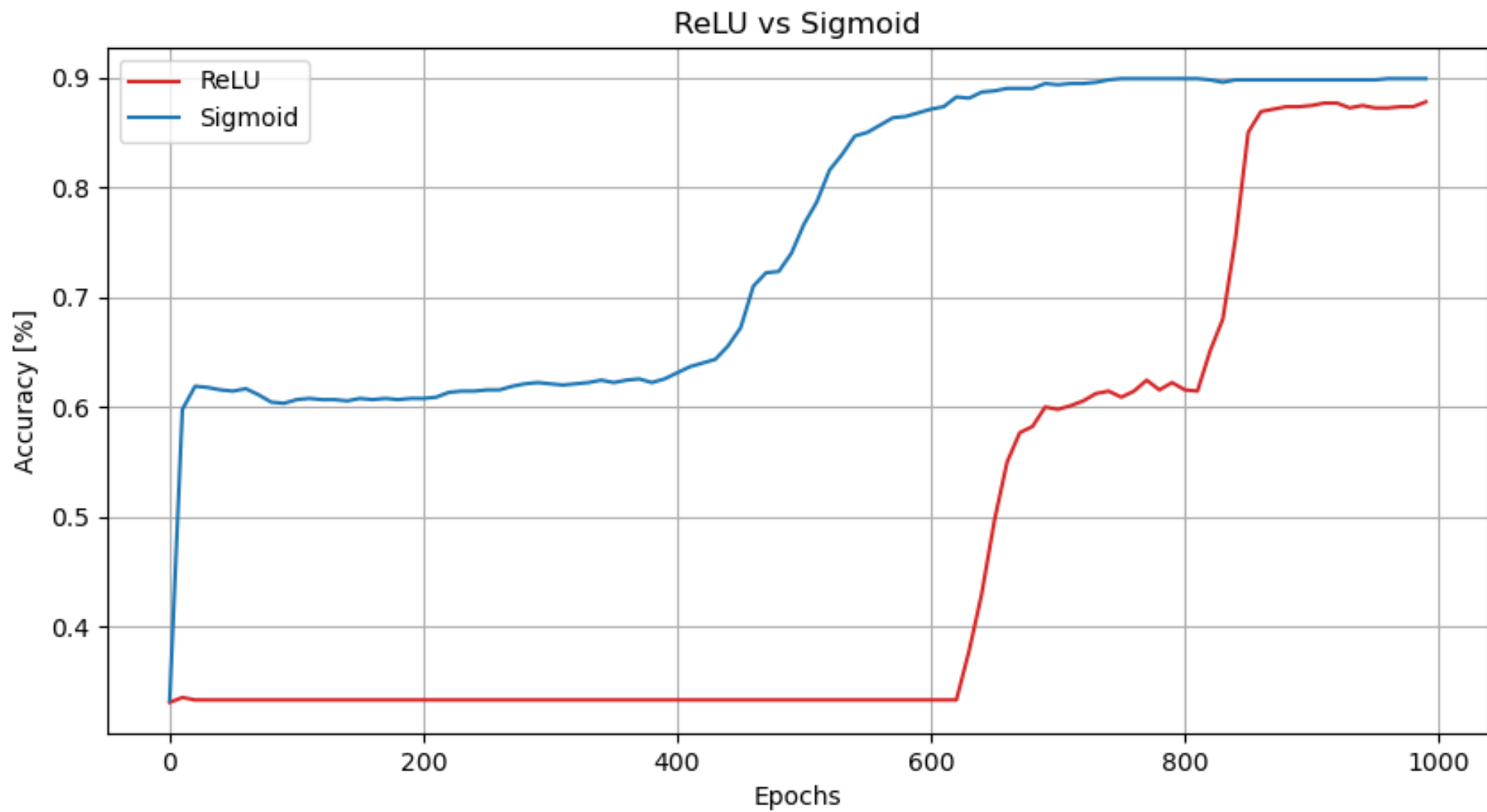
02 실습 #1 코드 분석

```
print(f'ReLU Accuracy: {accuracy_relu * 100:.2f}%')
print(f'Sigmoid Accuracy: {accuracy_sigmoid * 100:.2f}%')

plt.figure(figsize=(10, 5))
plt.plot(range(0, epochs, graph_scale), Accuracy_history_relu, label='ReLU', color='tab:red')
plt.plot(range(0, epochs, graph_scale), Accuracy_history_sigmoid, label='Sigmoid', color='tab:blue')
plt.xlabel('Epochs')
plt.ylabel('Accuracy [%]')
plt.title('ReLU vs Sigmoid')
plt.legend()
plt.grid()
plt.show()
```

그래프를 그리는 부분

02 실습 #1 실행 결과



ReLU VS Sigmoid

- 해당 가중치에서는 시그모이드의 성능이 더 좋은것으로 나옴
- 신경망이 깊어지면 깊어질수록 시그모이드의 성능은 감소할것으로 예측됨 (Gradient Vanishing 문제 때문)
- ReLU는 역전파되는 값 자체가 시그모이드보다 매우 크기에, 시그모이드 대비 더 작은 학습률 필요 (높은 학습률에서는 Dying ReLU 문제 발생 가능)
- ReLU는 코드화했을 때 Sigmoid대비 매우 간단하기 때문에, 계산 속도가 빠름
-> CNN같은 딥러닝 아키텍처에 많이 쓰임
- Sigmoid는 Dying ReLU와 같은 현상이 발생하지 않음
-> Binary Classification, Logistic Regression 등에서 자주 쓰임

03

실습 #2

Softmax & CEE

실습 #2) Deep Neural Network 구현

- Chap.1 실습 #2)에서 출력층을 제외한 모든 활성화 함수 자유
- Chap.1 실습 #2)에서 출력층 활성화 함수 Softmax로 변경
- Chap.1 실습 #2)에서 Loss function CEE로 변경
- Train set, Test set을 분할하여 사용
- 정확도 비교 이외에도 여러 분석 내용 기재

03 실습 #2 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

필요한 모듈 импорт

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_d(x):
    return x * (1 - x)
def ReLU(arr):
    return np.where(arr > 0, arr, 0)
def ReLU_d(arr):
    return np.where(arr > 0, 1, 0)
def cross_entropy_error(y, y_hat):
    return -np.mean(np.sum(y * np.log(y_hat + 0.0000001), axis=1))
def softmax(t):
    e_t = np.exp(t - np.max(t, axis=1, keepdims=True))
    return e_t / np.sum(e_t, axis=1, keepdims=True)
```

필요한 함수들을 정의한다.

softmax(): sigmoid와 비슷한데 총 합이 1이 되도록 함

cross_entropy_error(): CEE를 계산하는 함수

03 실습 #2 코드 분석

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

```
indices = np.arange(X.shape[0])
np.random.shuffle(indices)
split_ratio = 0.8
split_index = int(split_ratio * len(indices))
train_indices = indices[:split_index]
test_indices = indices[split_index:]
X_train, X_test = X[train_indices], X[test_indices]
y_train, y_test = y_one_hot[train_indices], y_one_hot[test_indices]
```

Training Set과 Test Set을 8:2로 나누는 과정

```
input_size = X.shape[1]
hidden_size = 5 # 히든 레이어의 노드 수
output_size = len(np.unique(y)) # 출력 클래스 수
learning_rate = 0.1
epochs = 10000
graph_scale = 10 # 그래프의 점을 얼마나 촘촘하게 찍을것인지, int (숫자가 작아질수록 촘촘함)
```

Hyper Parameter들을 정의

03 실습 #2 코드 분석

```
for epoch in range(epochs):
    # Forward Propagation
    layer_inputs = [X_train]
    layer_outputs = []
    # Forward Propagation during Hidden Layers
    for i in range(num_hidden_layers):
        input_to_layer = np.dot(layer_inputs[-1], weights[i])
        output_from_layer = ReLU(input_to_layer)
        layer_inputs.append(input_to_layer)
        layer_outputs.append(output_from_layer)
    # 출력 레이어로 전달
    final_input = np.dot(layer_outputs[-1], weights[-1])
    final_output = softmax(final_input)

    # 오차 구하기 & 그래프를 그리기 위한 밀작업
    loss = cross_entropy_error(y_train, final_output)
    error = y_train - final_output
    if epoch % 1000 == 0:
        print(f'Epoch {epoch}, Loss: {loss}')
    if epoch % graph_scale == 0:
        Loss_history.append(np.mean(np.abs(error)))
        y_pred = np.zeros_like(final_output)
        y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
        accuracy = np.mean(np.all(y_pred == y_train, axis=1))
        Accuracy_history.append(accuracy)

    # Backward Propagation
    d_output = error
    d_hidden = [d_output]
    for i in range(num_hidden_layers - 1, -1, -1): # for(int i=num....; i <= 0; i--) 이라는 의미
        d_hidden_layer = np.dot(d_hidden[0], weights[i + 1].T) * ReLU_d(layer_outputs[i])
        d_hidden.insert(0, d_hidden_layer) # 각 히든레이어들의 미분값 계산
    weights[-1] += np.dot(layer_outputs[-1].T, d_output) * learning_rate # 마지막 히든 -> 출력 사이의 가중치 업데이트
    for i in range(num_hidden_layers - 1, 0, -1):
        weights[i] += np.dot(layer_outputs[i - 1].T, d_hidden[i]) * learning_rate # 각 히든레이어 사이의 가중치 업데이트
    weights[0] += np.dot(X_train.T, d_hidden[0]) * learning_rate # 입력 -> 히든 사이의 가중치 업데이트
```

Chap.1의 실습 코드에서 활성화함수를 sigmoid로, 출력층 함수를 softmax로 바꾼 코드

03 실습 #2 코드 분석

```
for epoch in range(epochs):
    # Forward Propagation
    layer_inputs = [X_train]
    layer_outputs = []
    # Forward Propagation during Hidden Layers
    for i in range(num_hidden_layers):
        input_to_layer = np.dot(layer_inputs[-1], weights[i])
        output_from_layer = ReLU(input_to_layer)
        layer_inputs.append(input_to_layer)
        layer_outputs.append(output_from_layer)
    # 출력 레이어로 전달
    final_input = np.dot(layer_outputs[-1], weights[-1])
    final_output = softmax(final_input)

    # 오차 구하기 & 그래프를 그리기 위한 밀작업
    loss = cross_entropy_error(y_train, final_output)
    error = y_train - final_output
    if epoch % 1000 == 0:
        print(f'Epoch {epoch}, Loss: {loss}')
    if epoch % graph_scale == 0:
        Loss_history.append(np.mean(np.abs(error)))
        y_pred = np.zeros_like(final_output)
        y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
        accuracy = np.mean(np.all(y_pred == y_train, axis=1))
        Accuracy_history.append(accuracy)

    # Backward Propagation
    d_output = error
    d_hidden = [d_output]
    for i in range(num_hidden_layers - 1, -1, -1): # for(int i=num....; i <= 0; i--) 이라는 의미
        d_hidden_layer = np.dot(d_hidden[0], weights[i + 1].T) * ReLU_d(layer_outputs[i])
        d_hidden.insert(0, d_hidden_layer) # 각 히든레이어들의 미분값 계산
    weights[-1] += np.dot(layer_outputs[-1].T, d_output) * learning_rate # 마지막 히든 -> 출력 사이의 가중치 업데이트
    for i in range(num_hidden_layers - 1, 0, -1):
        weights[i] += np.dot(layer_outputs[i - 1].T, d_hidden[i]) * learning_rate # 각 히든레이어 사이의 가중치 업데이트
    weights[0] += np.dot(X_train.T, d_hidden[0]) * learning_rate # 입력 -> 히든 사이의 가중치 업데이트
```

Chap.1의 실습 코드에서 활성화함수를 ReLU로, 출력층 함수를 softmax로 바꾼 코드

03 실습 #2 코드 분석

```
epochs_range = range(0, epochs, graph_scale)
fig, ax1 = plt.subplots()

ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
loss_plot, = ax1.plot(epochs_range, Loss_history, color='tab:red', label='Loss')
ax1.tick_params(axis='y', labelcolor='tab:red')

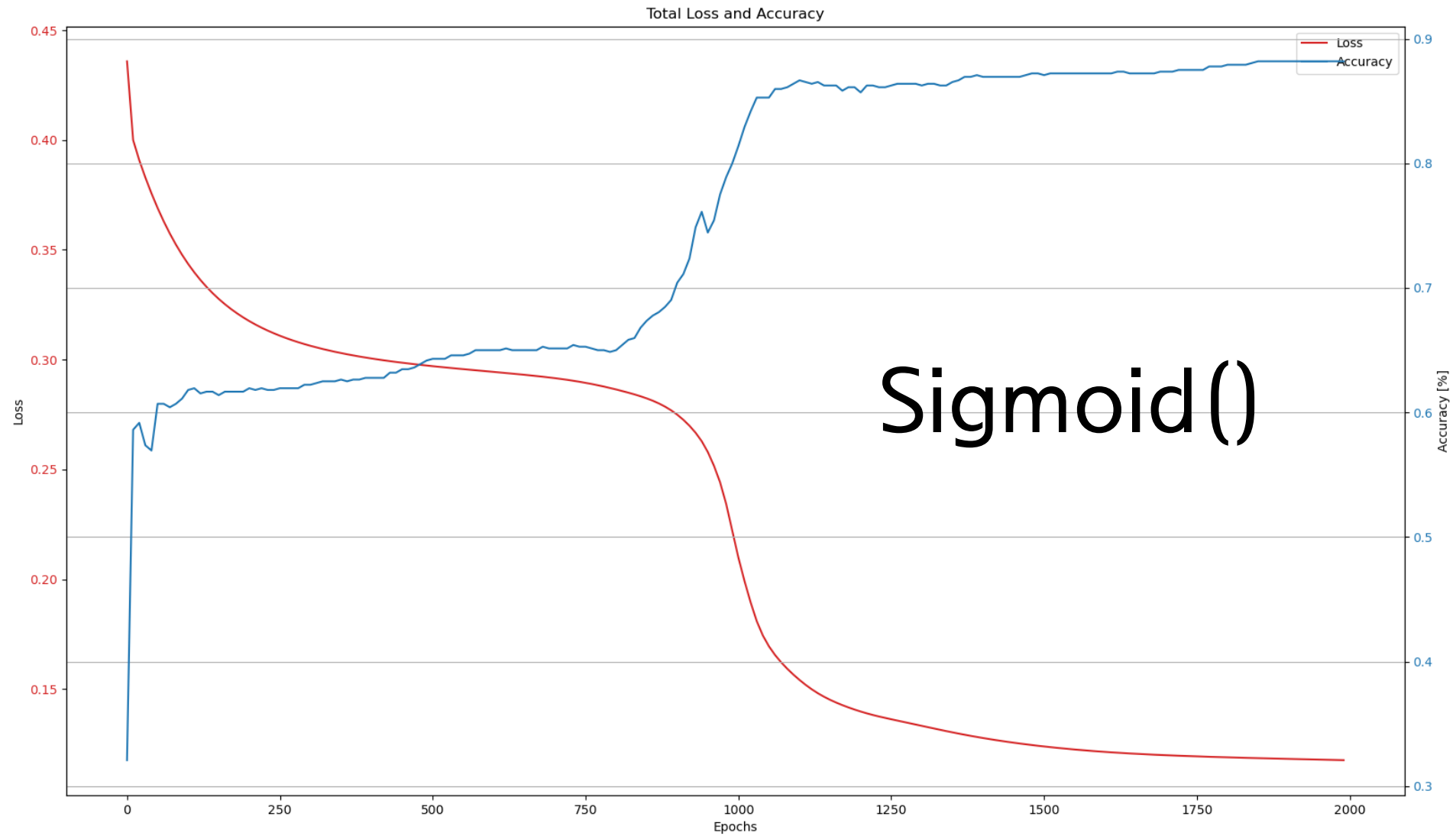
ax2 = ax1.twinx()
ax2.set_ylabel('Accuracy [%]')
accuracy_plot, = ax2.plot(epochs_range, Accuracy_history, color='tab:blue', label='Accuracy')
ax2.tick_params(axis='y', labelcolor='tab:blue')

fig.tight_layout()
plt.title('Total Loss and Accuracy')
plt.grid(True)
lines = [loss_plot, accuracy_plot]
labels = [line.get_label() for line in lines]
ax1.legend(lines, labels, loc=0)

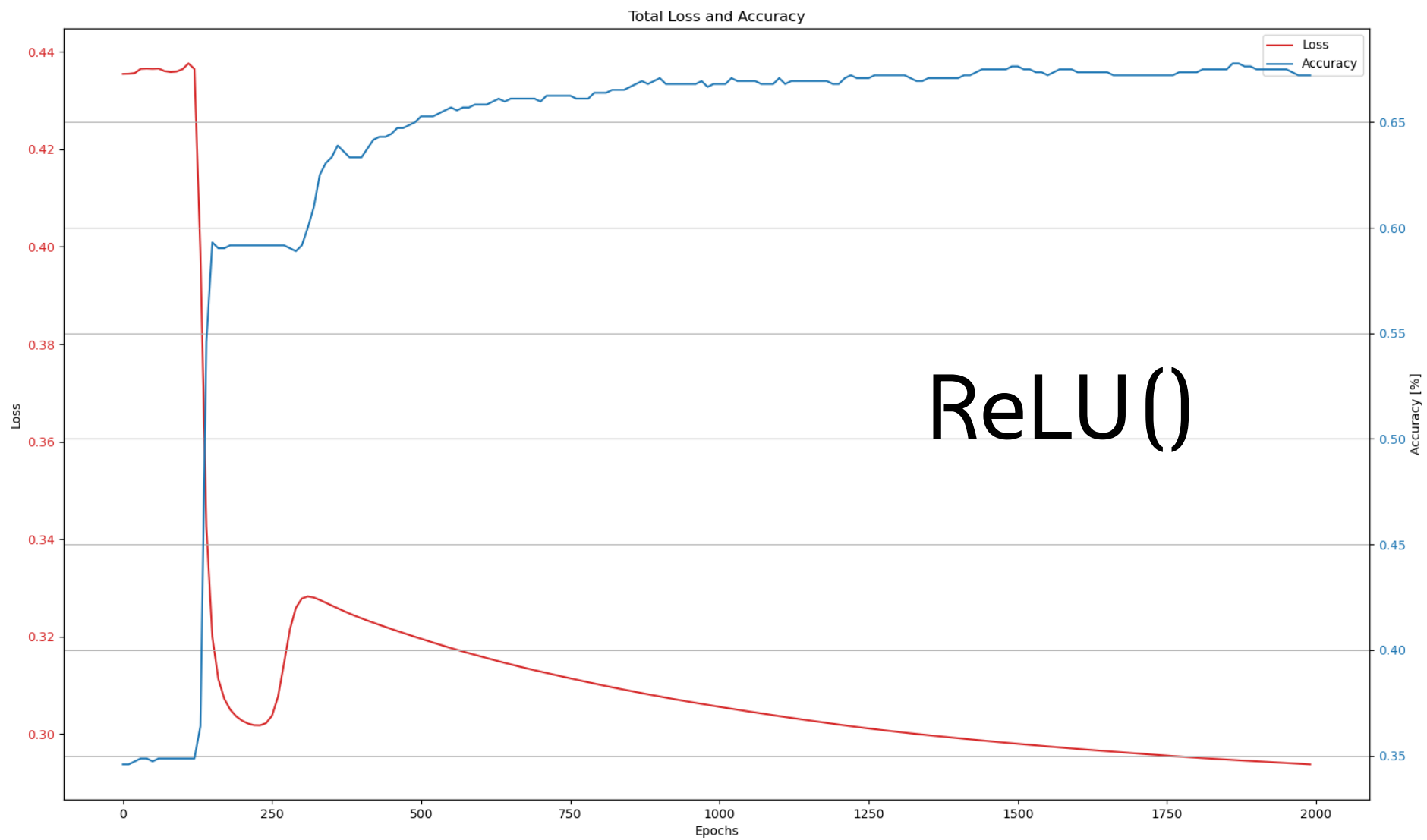
plt.show()
```

그래프를 그리는 부분

03 실습 #2 실행 결과



03 실습 #2 실행 결과



ReLU VS Sigmoid

- 해당 가중치에서는 ReLU의 성능이 더 좋은것으로 나옴
- 신경망이 깊어지면 깊어질수록 시그모이드의 성능은 감소할것으로 예측됨 (Gradient Vanishing 문제 때문)
- ReLU는 역전파되는 값 자체가 시그모이드보다 매우 크기에, 시그모이드 대비 더 작은 학습률 필요 (높은 학습률에서는 Dying ReLU 문제 발생 가능)
- ReLU와 Sigmoid의 적정 learning rate가 다르기 때문에, 한 쪽의 학습률을 그대로 다른 쪽에 적용하면 Overfitting / Underfitting 문제가 발생

04

실습 #3

DNN by keras

실습 #3)

keras를 활용하여 Deep Neural Network 구현

- 실습 예시에서 사용한 Dense 층을 활용하여 DNN 구현
- Chap.1에서 사용한 “NN_data.csv” 데이터 사용
- label 데이터는 One-Hot Encoding 하여 사용
- Train set, Test set을 분할하여 사용
- GPU vs CPU 사용에 따른 훈련 시간 비교
- Activation Function: Sigmoid vs ReLU vs LeakyReLU 간 비교

04 실습 #3 코드 분석

```
from tensorflow import keras
from keras.layers import Dense
from keras.models import Sequential
from keras.optimizers import SGD

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time # 실행 시간을 측정하기 위해 임포트
```

필요한 모듈 임포트

```
def One_Hot_Encoding(y, number_of_class):
    """원 - 핫 인코딩을 실행하는 함수"""
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot
```

필요한 함수를 정의한다.

나머지 함수들은 keras안에 잘 정의되어 있으므로 생략한다.

04 실습 #3 코드 분석

```
model_sigmoid = Sequential([
    Dense(units=128, activation='sigmoid', input_shape=(X_train.shape[1],)),
    Dense(units=128, activation='sigmoid'),
    Dense(units=128, activation='sigmoid'),
    Dense(units=64, activation='sigmoid'),
    Dense(units=num_classes, activation='softmax'),
])

model_relu = Sequential([
    Dense(units=128, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(units=128, activation='relu'),
    Dense(units=128, activation='relu'),
    Dense(units=64, activation='relu'),
    Dense(units=num_classes, activation='softmax'),
])

model_leakyrelu = Sequential([
    Dense(units=128, activation='leaky_relu', input_shape=(X_train.shape[1],)),
    Dense(units=128, activation='leaky_relu'),
    Dense(units=128, activation='leaky_relu'),
    Dense(units=64, activation='leaky_relu'),
    Dense(units=num_classes, activation='softmax'),
])
```

각각의 함수에 대한 모델들을 생성한다.

출력층의 활성화함수는 softmax이며, 각 레이어의 노드 수는 128, 64개로 하였다.

04 실습 #3 코드 분석

```
training_time = {}
# 모델 컴파일 및 학습
optimizer = SGD(learning_rate=0.3)
start_time = time.time()
model_sigmoid.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_sigmoid = model_sigmoid.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = time.time()
training_time['Sigmoid'] = end_time - start_time
print("Sigmoid 모델 학습 완료!\n")

optimizer = SGD(learning_rate=0.001)
start_time = time.time()
model_relu.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_relu = model_relu.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = time.time()
training_time['ReLU'] = end_time - start_time
print("ReLU 모델 학습 완료!\n")

optimizer = SGD(learning_rate=0.001)
start_time = time.time()
model_leakyrelu.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_leakyrelu = model_leakyrelu.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = time.time()
training_time['Leaky ReLU'] = end_time - start_time
print("Leaky ReLU 모델 학습 완료!\n")
```

모델들의 학습을 하는 부분

Optimizer로는 Stochastic Gradient Descent를 이용했고, epochs = 15, batch_size = 4로 설정했다.

04 실습 #3 코드 분석

```
Loss_history_sigmoid = history_sigmoid.history['loss']
Accuracy_history_sigmoid = history_sigmoid.history['accuracy']
Loss_history_relu = history_relu.history['loss']
Accuracy_history_relu = history_relu.history['accuracy']
Loss_history_leakyrelu = history_leakyrelu.history['loss']
Accuracy_history_leakyrelu = history_leakyrelu.history['accuracy']
```

Loss 및 정확도를 기록하는 부분

```
epochs_range = range(epochs)
fig, ax1 = plt.subplots()
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.plot(epochs_range, Loss_history_sigmoid, color='tab:red', label='Loss (Sigmoid)', linestyle=':')
ax1.plot(epochs_range, Loss_history_relu, color='tab:orange', label='Loss (ReLU)', linestyle=':')
ax1.plot(epochs_range, Loss_history_leakyrelu, color='tab:green', label='Loss (Leaky ReLU)', linestyle=':')
ax1.tick_params(axis='y', labelcolor='tab:red')
ax2 = ax1.twinx()
ax2.set_ylabel('Accuracy [%]')
ax2.plot(epochs_range, Accuracy_history_sigmoid, color='tab:blue', label='Accuracy (Sigmoid)')
ax2.plot(epochs_range, Accuracy_history_relu, color='tab:purple', label='Accuracy (ReLU)')
ax2.plot(epochs_range, Accuracy_history_leakyrelu, color='tab:cyan', label='Accuracy (Leaky ReLU)')
ax2.tick_params(axis='y', labelcolor='tab:blue')
fig.tight_layout()
plt.title('Total Loss and Accuracy for Different Activation Functions')
plt.grid()
lines = ax1.get_lines() + ax2.get_lines()
labels = [line.get_label() for line in lines]
ax1.legend(lines, labels, loc='upper left')
```

그래프를 그리는 부분

04 실습 #3 코드 분석

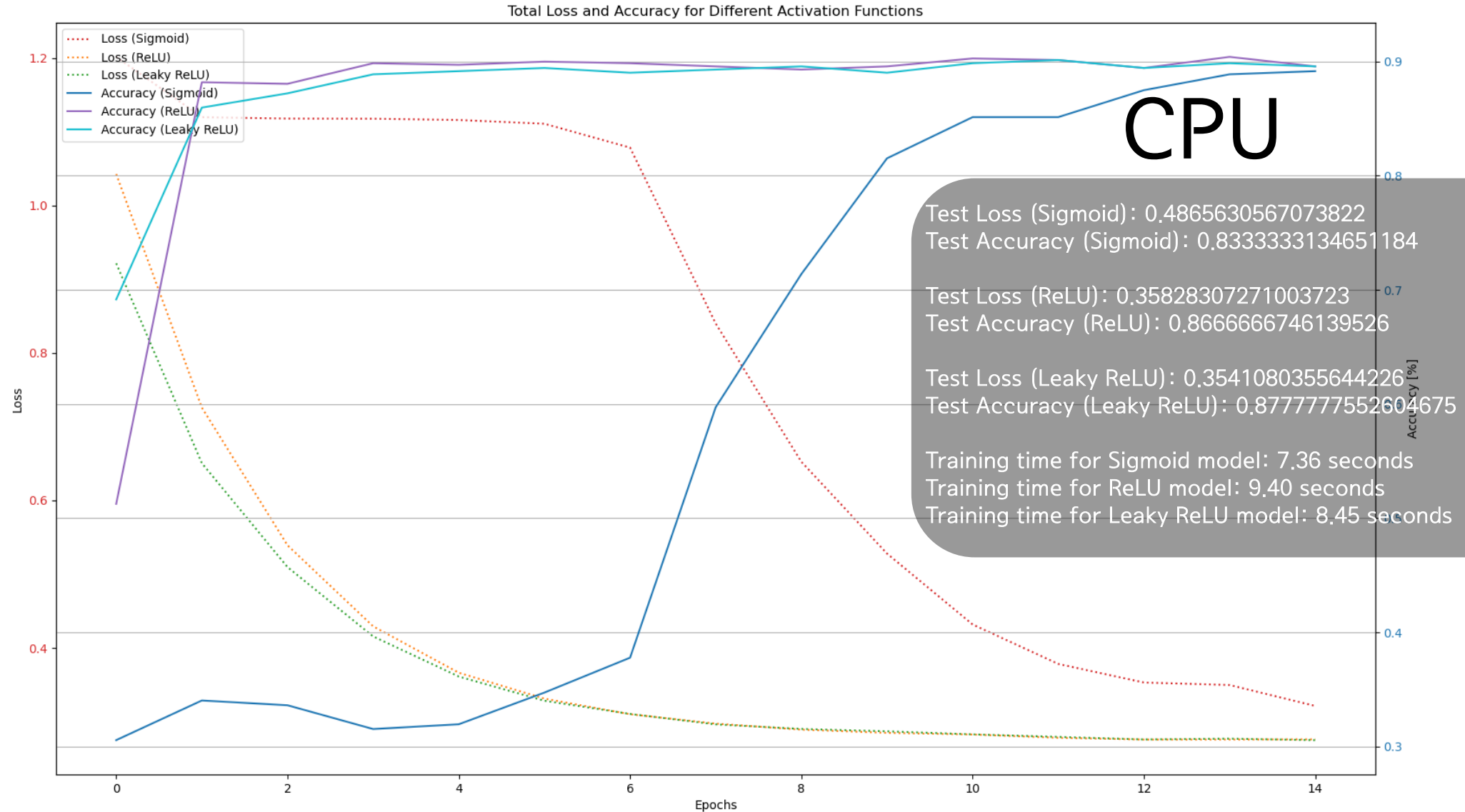
```
test_loss_sigmoid, test_accuracy_sigmoid = model_sigmoid.evaluate(X_test, y_test)
test_loss_relu, test_accuracy_relu = model_relu.evaluate(X_test, y_test)
test_loss_leakyrelu, test_accuracy_leakyrelu = model_leakyrelu.evaluate(X_test, y_test)

print(f'\nTest Loss (Sigmoid): {test_loss_sigmoid}\nTest Accuracy (Sigmoid): {test_accuracy_sigmoid}')
print(f'\nTest Loss (ReLU): {test_loss_relu}\nTest Accuracy (ReLU): {test_accuracy_relu}')
print(f'\nTest Loss (Leaky ReLU): {test_loss_leakyrelu}\nTest Accuracy (Leaky ReLU): {test_accuracy_leakyrelu}\n')

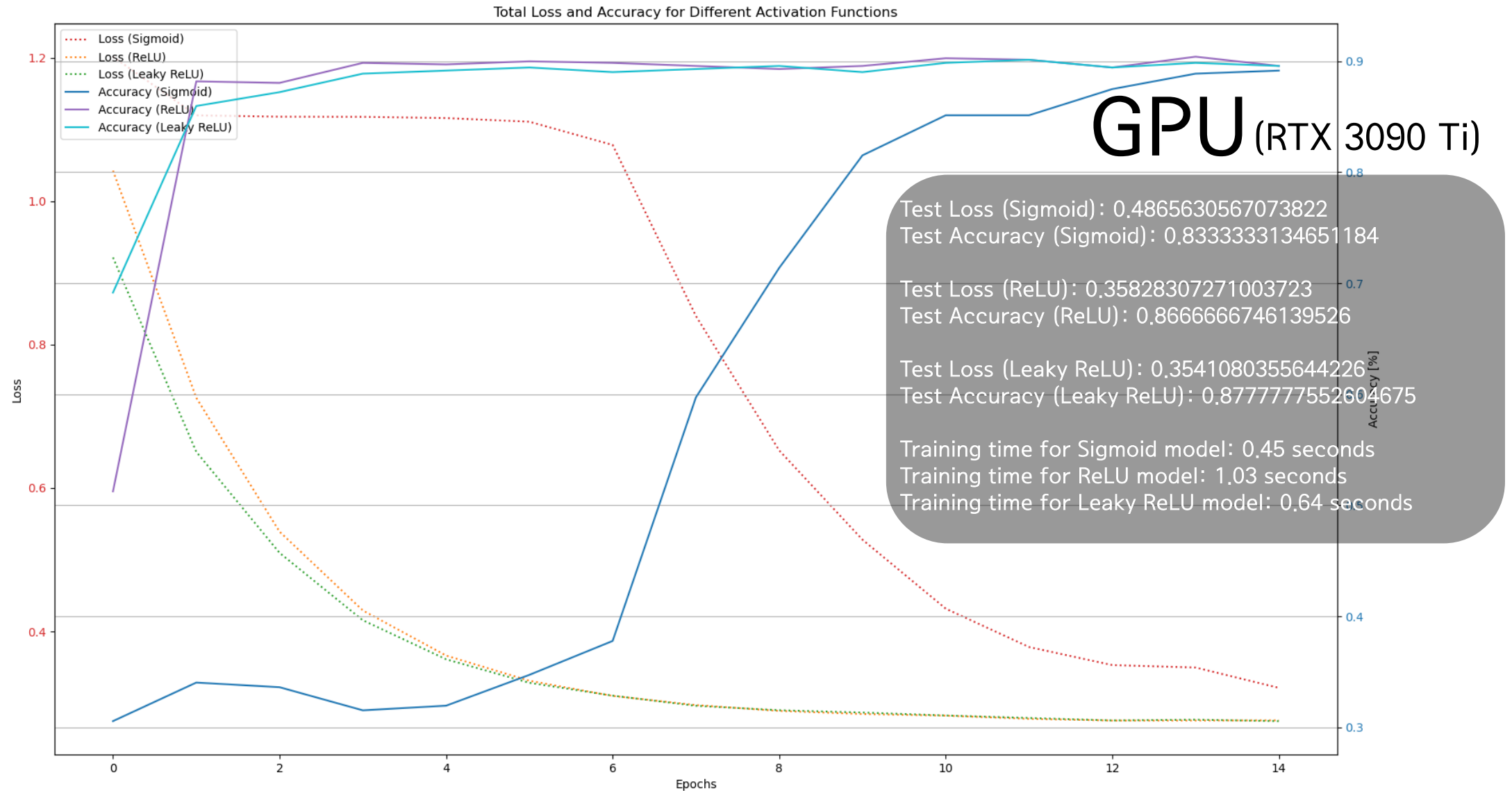
for model_name, duration in training_time.items():
    print(f'Training time for {model_name} model: {duration:.2f} seconds')
```

X_test와 y_test로 모델이 잘 학습되었는지 평가하는 부분

04 실습 #3 실행 결과



04 실습 #3 실행 결과



CPU VS GPU

- 해당 가중치에서는 Leaky ReLU > ReLU > Sigmoid 순으로 성능이 좋음
- CPU 대비 GPU가 훨씬 빠른 속도로 학습이 가능함
- batch size가 2의 거듭제곱수일때 가장 최적 (GPU가 한 번에 연산할 수 있는 양이 2의 배수개 이기 때문)
- 딥러닝에서는 CPU 대비 GPU가 압도적인 성능을 가짐

고급머신러닝 보고서
감사합니다